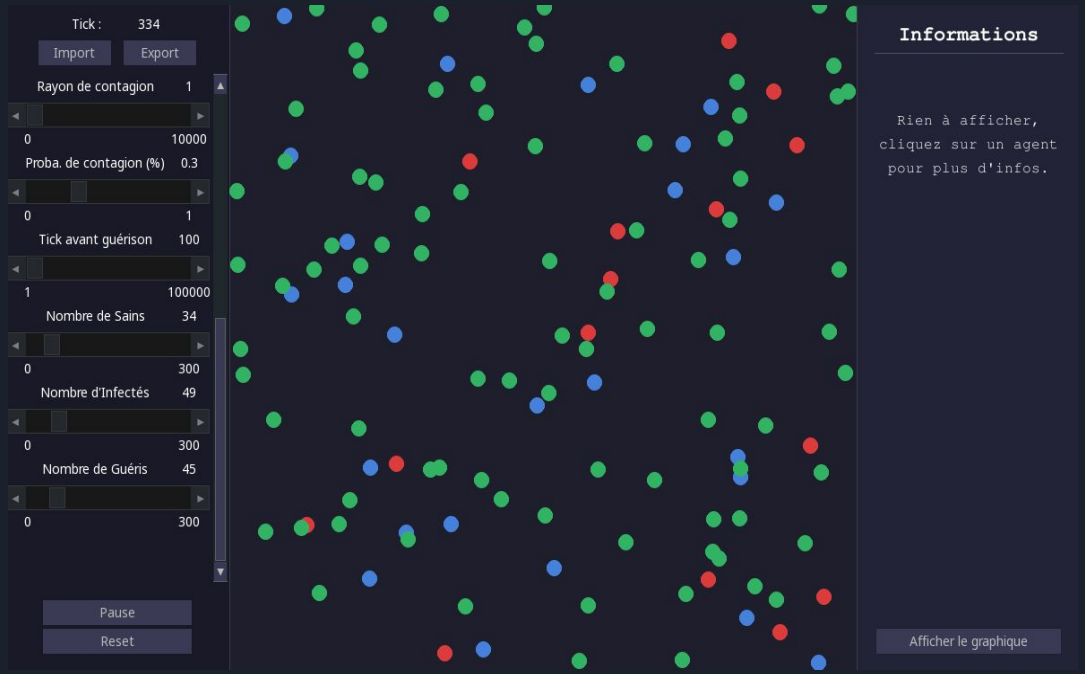
Henri CHIV
Michel CHEN

Projet long : ESCAS

(Epidemics and Social Contagion Agent-based Simulator)



Qui sommes nous?

Nous sommes des étudiants en Informatique, en M1 dans le master IMPAIRS à l'Université de Paris Cité.

L'objectif de notre projet:

Voir à travers la modélisation et la simulation les problèmes suivants:

- Comment modéliser la propagation d'un virus au sein d'une population ?
- En quoi l'ajout de comportement social et psychologique influe sur la propagation du virus ?
- Quel est le rôle de l'environnement dans cette propagation ?

Fonctionnalités et Cas d'usage

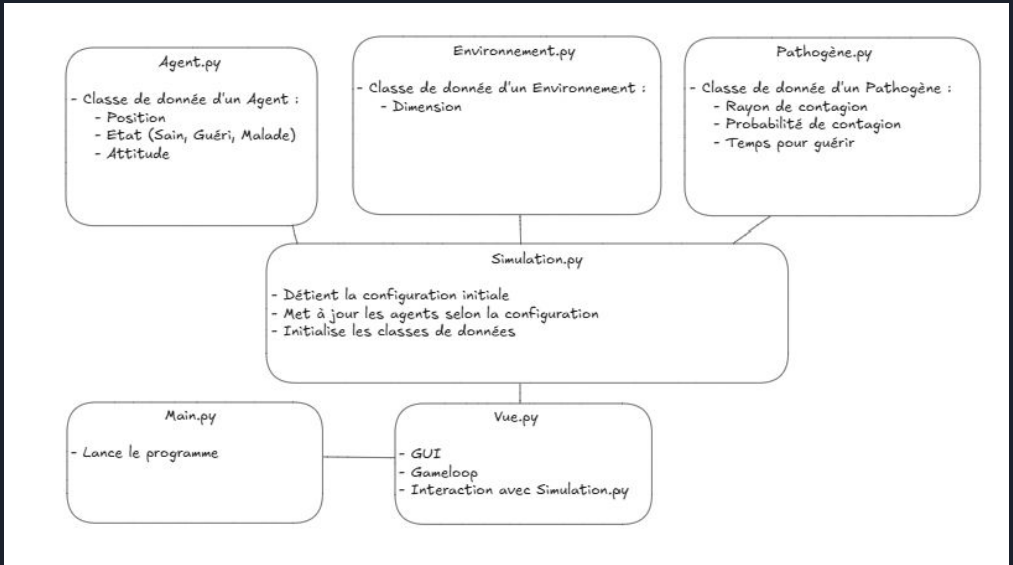
Les principaux scénarios d’utilisation:

- **Comparaison de virus:** COVID-19 vs Hantavirus.
- **Comportement social:** Effet d'une panique collective.
- **Sensibilisation:** Impact des gestes barrières et dangerosité d’un virus.
- **Analyse de données:** Analyse des graphes et des données d’un virus.

Ce que ESCAS permet de faire:

- **Simuler:** Générer une population d'agents autonomes dotés de comportements sociaux.
- **Observer:** Visualiser les infections et les déplacements des individus sur une map.
- **Manipuler:** Modifier en temps réel la biologie du virus et le nombre d’agents.
- **Analyser:** Suivre l'évolution des agents via un graphique temps réel.

Architecture générale



Agent-based Model (Représentation abstraite de la réalité en séparant les individus dans des groupes.)

Codé en Python.

Utilise Pygame et Pygame GUI pour l'interface graphique.

Choix de conception

Séparation claire des responsabilités entre les composants :

- La Vue se contente d'afficher les données de la Simulation
- La Simulation met à jour les agents selon la configuration donnée au départ

Ce qui permet d'ajouter de manière simple et rapide de nouvelles fonctionnalités tout en respectant les règles métiers.

Organisation et défis rencontrés

Organisation:

Recherche : ~2 semaines*
Logiciel : ~2 semaines*

Défis rencontrés:

- Données scientifiques pauvres/non trouvées
- Limites de l'ABM (explicité dans l'article scientifique)

Fragment de Code #1

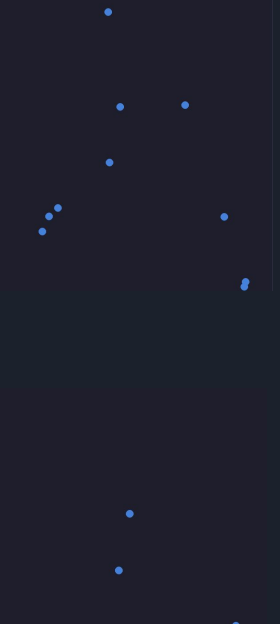
```python
def check_slider_move(self, event):
    for ctrl_id, slider in self.sliders.items():
        if event.ui_element == slider:
            # Variable pour mis à jour nombre d'agent
            old_nb_sains = self.config["nb_sains"]
            old_nb_infectes = self.config["nb_infectes"]
            old_nb_gueris = self.config["nb_gueris"]

            if ctrl_id == "proba_contagion":
                valeur = round(float(event.value), 2)
            else:
                valeur = int(event.value)
            self.config[ctrl_id] = valeur
            self.valeurs_labels[ctrl_id].set_text(str(valeur))

            if ctrl_id == "seed" or ctrl_id == "largeur" or ctrl_id == "hauteur":
                self.sim.reset(self.config)
            elif ctrl_id == "nb_sains":
                self.sim.modify_agents_nb(self.config[ctrl_id] - old_nb_sains, 0, 0)
            elif ctrl_id == "nb_infectes":
                self.sim.modify_agents_nb(0, self.config[ctrl_id] - old_nb_infectes, 0)
            elif ctrl_id == "nb_gueris":
                self.sim.modify_agents_nb(0, 0, self.config[ctrl_id] - old_nb_gueris)
```







```python
def modify_agents_nb(self, nb_sains, nb_infectes, nb_gueris):
    (nb_sains, nb_infectes, nb_gueris) = self.show_agents(nb_sains, nb_infectes, nb_gueris)
    self.unshow_agents(nb_sains, nb_infectes, nb_gueris)

def show_agents(self, nb_sains, nb_infectes, nb_gueris):
    while nb_sains > 0 and self.sains_shown < len(self.sains):
        self.sains[self.sains_shown].shown = True
        self.sains_shown += 1
        nb_sains -= 1
    while nb_infectes > 0 and self.infectes_shown < len(self.infectes):
        self.infectes[self.infectes_shown].shown = True
        self.infectes_shown += 1
        nb_infectes -= 1
    while nb_gueris > 0 and self.gueris_shown < len(self.gueris):
        self.gueris[self.gueris_shown].shown = True
        self.gueris_shown += 1
        nb_gueris -= 1
    return (nb_sains, nb_infectes, nb_gueris)

def unshow_agents(self, nb_sains, nb_infectes, nb_gueris):
    while nb_sains < 0 and self.sains_shown > 0:
        self.sains[self.sains_shown - 1].shown = False
        self.sains_shown -= 1
        nb_sains += 1
    while nb_infectes < 0 and self.infectes_shown > 0:
        self.infectes[self.infectes_shown - 1].shown = False
        self.infectes_shown -= 1
        nb_infectes += 1
    while nb_gueris < 0 and self.gueris_shown > 0:
        self.gueris[self.gueris_shown - 1].shown = False
        self.gueris_shown -= 1
        nb_gueris += 1
```

Fragment de Code #2

```python
def update_contagion(self):
    nouveaux_infectes = []

    for agent in self.agents:
        if not agent.shown: continue
        agent.tick_etat += 1
        if (agent.etat == Etat.INFECTE):
            temps_cible = getattr(agent, 'tick_guerison_cible', self.pathogene.tick_guerison * getattr(agent, 'healing_factor', 1.0))
            # Guérison
            if (agent.tick_etat >= temps_cible):
                agent.etat = Etat.GUERI
                agent.tick_etat = 0
            else:
                # Infection des voisins
                for voisin in self.agents:
                    if voisin.etat == Etat.SAIN:
                        dist = (voisin.x - agent.x)**2 + (voisin.y - agent.y)**2
                        if dist <= self.pathogene.rayon_contagion**2:
                            proba = self.pathogene.proba_contagion
                            if voisin.attitude == Attitude.PANIQUE:
                                proba *= 1.3
                            elif voisin.attitude == Attitude.CONFINE:
                                proba *= 0.4

                            if (random.random() < proba):
                                nouveaux_infectes.append(voisin)


    for agent in nouveaux_infectes:
        agent.etat = Etat.INFECTE
        agent.tick_etat = 0
        facteur = getattr(agent, 'healing_factor', 1.0)
        agent.tick_guerison_cible = self.pathogene.tick_guerison * facteur
```

Limites actuelles du logiciel

Par manque de temps et d'organisation, nous n'avons pas pu implémenter ces fonctionnalités, on peut donc ainsi les implémenter dans la version 2 du logiciel:

- Comparaison avec modèle SIR
- Export des graphes au format PNG.
- Profil social (Interactions sociales)
- Utiliser la Méthode Monte-Carlo pour avoir des résultats fiables.
- Ajout de différentes façons d'infecter les agents.
- Ajout de "routines" pour les agents.

Ce qu'on aurait changé si on devait refaire le projet:

- Commencer le projet plus tôt (Recherche, code, test, etc…).
- Mieux s'organiser avec les autres projets.
- Faire un projet beaucoup plus long avec beaucoup plus de fonctionnalités.
- Se focaliser sur le contenu et les fonctionnalités plutôt que la structure et la forme.

Bibliographie / Références :

Agent-Based Modeling in Public Health: Current Applications and Future Directions (Melissa Tracy, Magdalena Cerdá, and Katherine M. Keyes)
https://en.wikipedia.org/wiki/Compartmental_models_(epidemiology)
https://en.wikipedia.org/wiki/Agent-based_model
https://fr.wikipedia.org/wiki/Automate_cellulaire
https://en.wikipedia.org/wiki/Social_contagion
https://fr.wikipedia.org/wiki/Intelligence_distribu%C3%A9e

Inspiration :
https://www.youtube.com/@Fouloscopie
https://www.youtube.com/@carykh